

Centrality-Based KV-Cache Eviction: Training-Free Personalized-PageRank Reinforcement for NVIDIA kvpress

An enhanced cache policy for NVIDIA kvpress.

Base: github.com/NVIDIA/kvpress · **Our work:** github.com/lapidoty/kvpress-centrality

1. Introduction

Large language models spend most of their long-context inference budget storing the key-value (KV) cache: every attention layer keeps a key and value vector for every past token. At 32k+ context this dominates memory and bandwidth. *KV-cache compression* discards the least useful entries during prefill, trading a little accuracy for large memory/throughput gains.

The baseline library for this work is **NVIDIA kvpress**, a mature, actively maintained framework (Apache-2.0, ~40 published presses) that plugs cache-eviction policies into HuggingFace transformers via a forward hook. Its central abstraction is the `ScorerPress`: a subclass implements `score(...)` $\rightarrow$ (batch, num_kv_heads, seq_len) and the framework keeps the top-scoring tokens. kvpress was chosen because it is the de-facto standard harness (leaderboard, RULER/LongBench evaluators, a tiny CPU unit-test model, an active PR history that merges external and even AI-authored scorers), so a new press can be benchmarked against many strong baselines under identical workloads with minimal glue.

Most eviction scorers rank each token *independently* (e.g. by key norm, or by attention from recent queries). They ignore the **relational structure** of the cache: a token can be individually unremarkable yet crucial because it *supports* an important token (a corroborating fact, a link in a multi-hop chain). This project adds a press that scores tokens by their **centrality in the key-similarity graph**, so importance can *propagate* from a token to its neighbours.

Naive centrality has two well-known failure modes for eviction: (i) *spectral localization*, the leading eigenvector piles onto the single densest cluster, keeping near-duplicates; and (ii) *outlier eviction*, a “needle”/answer token is by construction dissimilar to the haystack, so it has low centrality and is dropped (exactly the tokens `LeverageScorePress` deliberately keeps). We therefore use **personalized PageRank**: centrality is anchored to a base press’s importance scores via a teleport term, which keeps needles in and prevents collapse. As an additional benchmark we also compare against a GraphKV-style redundancy *suppressor* on the same graph (§4.6; code in `project/additional_benchmarks/`).

Contributions. (1) `CentralityPress`, a training-free personalized-PageRank eviction scorer with an $O(S \cdot \text{head_dim})$ low-rank kernel; (2) a paired, statistically-tested benchmark across RULER and LongBench, against baselines including a GraphKV-style suppressor; (3) a reproducible pack (one-command benchmark, sweep + analysis scripts, committed run logs).

Related work. kvpress ships many independent-scoring `ScorerPresses`: `KnormPress` (keep low-L2-norm keys), `SnapKVPress` (recent-query attention), `ExpectedAttentionPress`, `TOVAPress`, `PyramidKVPress`, and the leverage-score family (`LeverageScorePress/CompactorPress/CURPress`) that explicitly *keeps outliers*. The closest relational method is **GraphKV** (arXiv:2509.00388), which *removes* redundancy on the key graph. Our press is its *reinforcement* dual, it propagates importance along the graph, using a personalized-PageRank teleport to avoid the classic outlier-eviction and spectral-localization pathologies of raw eigenvector centrality.

2. Extension design

2.1 CentralityPress (reinforcement)

Let $K \in \mathbb{R}^{S \times D}$ be the (per batch, per kv-head) key matrix and K_n its L2-normalized rows. Define the similarity graph $A[i, j] = (1 + \cos(k_i, k_j))/2 \in [0, 1]$ (the “shifted” kernel; non-negative). Personalized PageRank iterates

$$c \leftarrow \text{damping} \cdot \text{normalize}(Ac) + (1 - \text{damping})p \quad \text{where} \quad p = \text{softmax}(\text{base_score} / \tau)$$

starting from $c_0 = p$, for `num_iters` steps; the final c is the keep-score.

- **p (teleport)** anchors centrality to a base press (the API default is `None` = uniform teleport = pure centrality; `KnormPress` is the *recommended* base and is used throughout the experiments). We chose `KnormPress` as the base because it is **keys-only** (query/attention-free, matching this press’s design), **essentially free** (Fig S1), the framework’s default scorer, and it yields a

smooth teleport, SnapKV’s recent-window score spike instead degenerates the teleport (§5). It keeps base-important tokens in (including outlier needles) and re-injects mass every step so it cannot fully localize.

- **damping = 0** gives $c = p$, so the keep *ranking* is the base press’s (a safe floor). With the default sink/window force-keep the kept set differs from the base only by those protected tokens, so the empirical floor sits within noise of the base ($\Delta \approx 0$, not significant, see §4.1). **damping = 1** gives pure eigenvector centrality (the failure-mode ablation).
- The first n_{sink} and last recent_window tokens are force-kept (attention sinks + recent window), as in every strong press.

Efficiency. The shifted/linear kernel is low rank ($A = K_n K_n^\top$, $\text{rank} \leq \text{head_dim}$), so a matvec never materializes the $S \times S$ graph:

$$Ac = 0.5 \sum c + 0.5 K_n (K_n^\top c) \Rightarrow O(S \cdot \text{head dim}) \text{ time and memory}$$

The scorer is therefore **query-free, attention-free, flash-attention compatible, and linear in sequence length**.

3. Experimental setup

3.1 Model, hardware, environment

- **Model:** meta-llama/Llama-3.1-8B-Instruct (bf16), 32 layers, 32 query / 8 KV heads (GQA), head_dim 128, the kvpress evaluate.py default.
- **Hardware:** one NVIDIA A100-80GB.
- **Stack:** Python ≥ 3.10 ; torch, transformers, datasets, kvpress 0.5.4, installed with `pip install -e ".[eval]"` (versions pinned by kvpress’s pyproject.toml).

3.2 Benchmarks and metric

- **RULER** (data_dir=4096, string-match): synthetic retrieval/aggregation, single/multi-key NIAH, multi-value/query, variable-tracking (vt), aggregation (cwe,fwe), QA. **All 13 tasks $\times$ 500 examples** are used.
- **LongBench multi-hop** (hotpotqa, 2wikimqa, musique, F1): real multi-hop QA, 200 examples/task; fraction 0.35 ($n=210$ total).

Metrics & workloads. Two axes. (i) *Eviction quality*, per-task accuracy (RULER string-match, LongBench F1). Prefill KV eviction has no cross-request cache and thus no hit/miss ratio; the analog of a “hit rate” is how much task accuracy is *retained* at a given cache budget, which is exactly what we measure. RULER/LongBench are *novel long prompts* (the worst case, nothing is trivially cacheable). (ii) *Systems cost*, KV-cache memory, prefill overhead, and decode latency (mean/p95/p99) and throughput, measured on a long-prompt workload (8192-token prompts, 128 decode steps, batch 1 and 8), reported in §4.4.

How accuracy is scored (per example). Generation is greedy (deterministic); each example provides a context, question, answer_prefix, and per-task max_new_tokens, and the pipeline decodes an answer over the (compressed) cache. We reuse kvpress’s own metric functions, applied per example:

- **RULER, substring match.** After stripping control characters and normalizing whitespace, most tasks use `string_match_all` = the fraction of gold reference strings found as a *case-insensitive substring* of the prediction (a single-needle example scores 1 if the gold string appears, else 0; a k-item task such as multi-value scores found/k); the qa_* tasks use `string_match_part` = 1 if *any* gold answer is a substring. (evaluation/benchmarks/ruler/calculate_metrics.py.)
- **LongBench, token-F1.** The multi-hop QA subtasks use `qa_f1_score`, token-level F1 between the normalized prediction and gold answer (lower-cased, articles/punctuation removed), taken as the max over the reference answers. (evaluation/benchmarks/longbench/calculate_metrics.py.)

A task’s accuracy is the mean $\times 100$ over its examples; the headline **macro** is the mean over all 13 tasks (the kvpress-leaderboard metric). **All results in §4.1 are produced by kvpress’s own evaluation harness**, evaluation/evaluate.py drives generation and benchmarks/ruler/calculate_metrics.py scores it, with our press registered in evaluate_registry.py.

3.3 Presses, ablation ladder, and correctness

Baselines: no_press (ceiling), KnormPress, SnapKVPress. Ablation ladder (same base press): base $\rightarrow$ suppression (graphkv_knorm) $\rightarrow$ pure centrality (centrality_pure, damping=1) $\rightarrow$ personalized PageRank (centrality_ppr_knorm, sweeping damping $\in \{0, 0.15, 0.3, 0.5, 0.85\}$), plus a standardized-teleport SnapKV variant. Compression ratios 0.25 / 0.5 / 0.75.

Correctness is anchored by 22 unit tests including: exact low-rank $\leftrightarrow$ explicit matvec, the damping=0 floor recovering the base kept-set (with sink/window off), a GPU-gated peak-memory linearity test, and a *thesis test* (personalization keeps an outlier needle that pure centrality drops). The CPU subset runs in CI.

4. Results

4.1 RULER, all 13 tasks (kvpress library harness, macro %)

Scored end-to-end by kvpress’s own evaluate.py + calculate_metrics. Macro = mean over all 13 tasks (the leaderboard metric). Every number in this section is the **fraction-0.06 screen** (≈ 30 examples/task, $n \approx 390$); no_press ceiling = **95.25** (run no_press__0.00__fraction0.060).

method	c=0.25	c=0.5	c=0.75
no_press (full cache)	95.25	95.25	95.25
knorm (base)	76.5	51.6	29.9
snarkv	82.3	69.9	40.5
centrality_ppr_knorm d=0.15	94.5	82.4	58.3
centrality_pure (d=1)	28.2	10.4	5.1

On the full 13-task set, centrality_ppr_knorm d=0.15 **significantly beats its own base (knorm) and SnapKV at every compression ratio** (the head-to-head vs the GraphKV suppressor is in §4.6), and at 25 % compression nearly matches full cache (94.5 vs 95.25). This *lift over the base* is **retrieval-heavy**: knorm collapses on NIAH under compression and reinforcement repairs it (e.g. NIAH-multikey 13.1 $\rightarrow$ 69.4 at c=0.5). damping=0 reproduces the base (floor); damping ≥ 0.5 and pure centrality collapse (pure = 28.2 / 10.4 / 5.1 macro).

Where this lands on the public leaderboard. Ranked against the Llama-3.1-8B methods the KVPress board evaluates at each ratio (full-fraction macro; analysis/rank_vs_board.py), reinforcement lifts the base press from near the bottom of the field to mid-pack:

compression	knorm (base)	1. centrality	Δ	rank jump
0.25	76.3	94.6	+18.3	19th $\rightarrow$ 8th
0.50	52.8	81.3	+28.5	20th $\rightarrow$ 12th
0.75	30.3	58.0	+27.7	20th $\rightarrow$ 13th
0.875	21.2	41.7	+20.5	19th $\rightarrow$ 11th

Mid-pack overall; the full 21-method board ranking (all four ratios) is in the supplementary (§S.4).

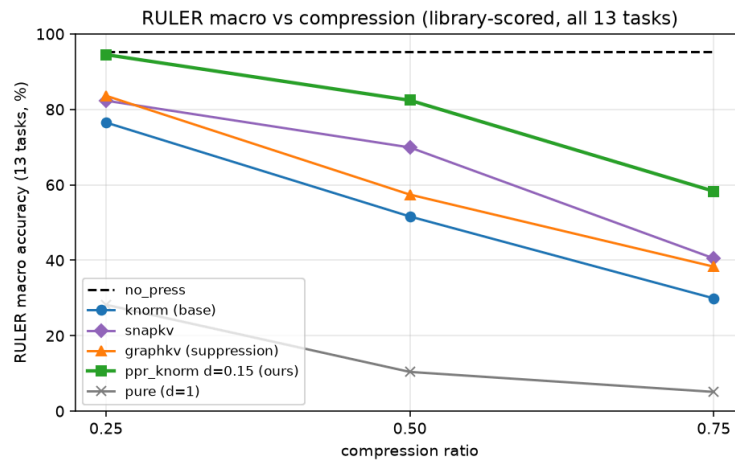


Figure 1: RULER macro accuracy (all 13 tasks, kvpress library scorer, **fraction 0.06 screen**) vs. compression. ppr_knorm d=0.15 (ours) dominates its base and the other baselines shown; not a leaderboard-eligible comparison.

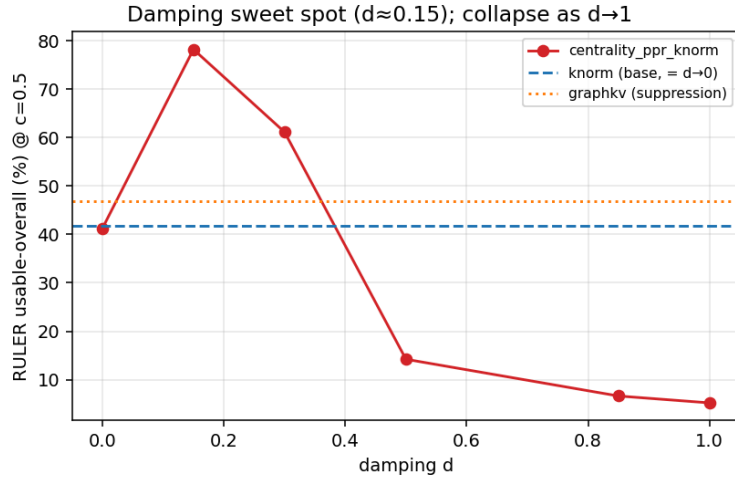


Figure 2: Damping sweep at $c=0.5$. A shallow reinforcement ($d \approx 0.15$) peaks above the base press; as $d \rightarrow 1$ (pure centrality) it collapses; $d \rightarrow 0$ recovers the base.

4.2 LongBench, multi-hop QA (F1 %, $n=210$)

method	c=0.5	c=0.75
no_press	47.4	47.4
knorm (base)	39.8	31.8
snapkv	45.1	n/a
centrality_ppr_knorm $d=0.15$	42.0	39.9

On real multi-hop QA the reinforcement benefit **grows with compression**: at mild 0.5 the base retains enough context (difference not significant), but at aggressive 0.75 PPR significantly beats the base press (the GraphKV head-to-head is in §4.6).

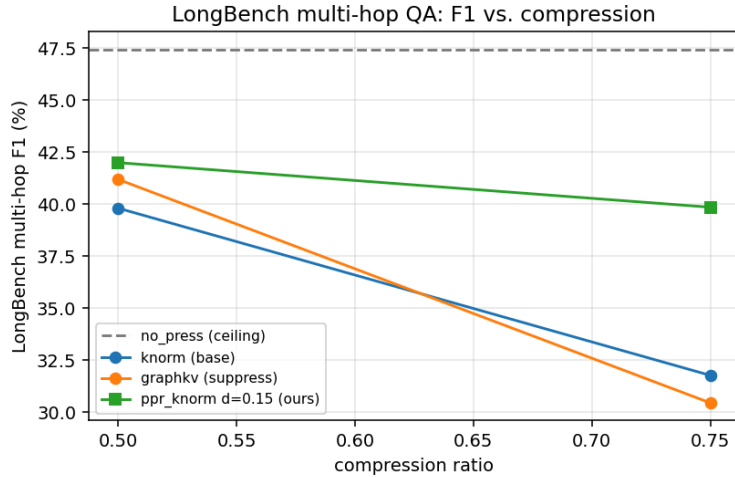


Figure 3: LongBench multi-hop F1 vs. compression. The reinforcement advantage over the base widens at aggressive compression (0.75).

4.3 Hypotheses

H1, that a training-free centrality press is viable and $O(S \cdot \text{head_dim})$ efficient, is established by construction and the linear-kernel analysis in §2–§3. The reinforcement-vs-suppression hypotheses (H2, H3) are stated and tested in §4.6.

4.4 Systems: memory, latency, throughput, GPU utilization

We measured the systems metrics across **all** presses (mirroring the accuracy sweep) on a long-prompt workload: 8192-token prompt, 128 decode steps, **batch 8**, Llama-3.1-8B bf16, A100-80GB. The central fact: **at a fixed compression ratio the kept-**

cache length is identical for every press, so decode memory, latency, throughput and GPU utilization are *ratio-determined*, the same for knorm, snapkv, centrality_pure and our press. The **only press-dependent systems cost is prefill overhead**.

Full comparison at ratio 0.5 (batch 8):

press	prefill (s)	peak GPU (GB)	KV cache (MB)	tok/s	latency mean/ p95/p99 (ms)	GPU util
no_press (r=0)	5.88	39.1	8192	180	44.4 / 48.6 / 54.6	75 %
knorm	6.07	35.1	4096	178	44.9 / 50.8 / 55.5	56 %
snapkv	6.21	35.1	4096	169	47.2 / 58.2 / 65.6	54 %
centrality_pure	6.06	35.1	4096	172	46.5 / 58.8 / 65.8	57 %
ppr_knorm d0.15	6.05	35.1	4096	173	46.4 / 56.2 / 60.8	55 %
ppr_snapkv_std d0.15	6.25	35.1	4096	166	48.2 / 66.7 / 74.0	53 %

Every press at $r=0.5$ uses the same 4096 MB / 35.1 GB and runs at ~the same latency and throughput (Figs S2–S3), memory and speed are functions of the *ratio*, not the policy. What differs is prefill overhead (Fig S1): **ppr_knorm (+0.18 s over no_press) is as cheap as knorm (+0.19 s)**, the low-rank kernel makes the PageRank iterations essentially free, and *cheaper* than every SnapKV-based press (+0.33–0.39 s, attention compute); its prefill edge over the GraphKV suppressor is noted in §4.6. Across the ratio sweep, peak memory falls **39.1 → 37.1 → 35.1 → 33.1 GB** and GPU utilization **75 → 64 → 56 → 45 %** as compression rises $0 \rightarrow 0.75$ (less attention compute per token), while decode throughput stays ~flat (~170–185 tok/s); at 8B/8k decode is weight-bound, so the latency/throughput benefit would grow at longer context or larger batch, where the KV cache dominates decode.

Cache hit rate is not defined for prefill KV eviction, there is no cross-request cache, hence no hit/miss; the meaningful analog is **retention accuracy** (Fig 4): how much task accuracy survives at a given cache budget. *CPU utilization* is negligible, the workload is GPU-bound.

Per-press systems figures, prefill overhead, decode-latency percentiles, memory/throughput and GPU utilization vs. ratio, are in the Supplementary Material (Figs S1–S4).

Because memory is fixed by the ratio, a better policy buys **more accuracy per byte**, the accuracy–memory Pareto front (Fig 4), where our press dominates the base press at every budget.

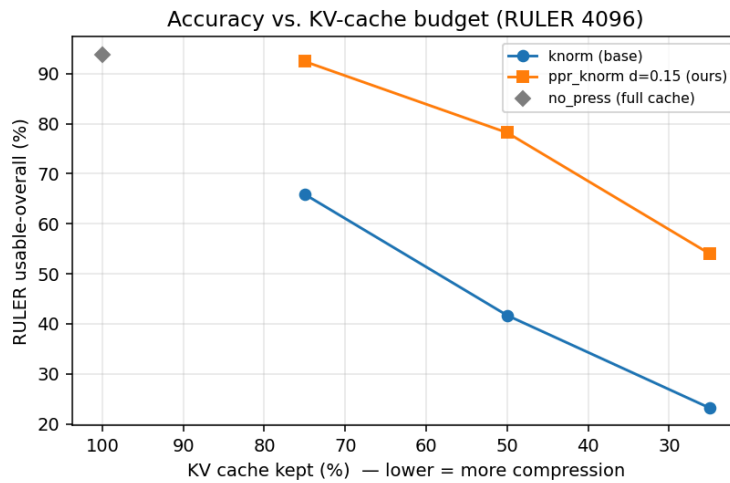


Figure 4: Accuracy vs. KV-cache budget on RULER (lower x = more compression), the retention / “hit-rate” analog. At every budget **ppr_knorm d=0.15** sits far above the base press at the same memory.

4.5 Iso-accuracy: memory, throughput and latency at equal quality

The systems metrics are set by the kept-cache length (§4.4) and the accuracy win is at a fixed ratio (§4.1); the two combine into the operationally decisive question, **for the same accuracy, how much does the smaller cache save?** We invert the RULER accuracy-vs-budget curves to the cache fraction each press needs to hit a target accuracy, then read the **directly measured** decode metrics at those fractions (16k-token prompt, batch 8, 128 decode steps, the KV-bound regime; see `results/results_iso_systems_16k_bs8.csv`, `analysis/iso_systems.py`).

target acc.	cache reduction	KV memory saved	peak GPU mem saved	decode throughput	decode p50 latency
40 %	4.5×	78 %	16 % (6.0 GB)	+1 %	−0 %
50 %	2.8×	64 %	16 % (6.0 GB)	−5 %	−6 %
60 %	2.2×	55 %	15 % (6.0 GB)	+3 %	+1 %
70 %	1.9×	47 %	14 % (6.0 GB)	+3 %	+3 %
80 %	1.6×	39 %	13 % (5.5 GB)	+3 %	+9 %

The iso-accuracy win is a memory win. At equal output quality our press needs **1.6×–4.5× less KV cache** (39–78 % less KV memory) and **~13–16 % (~6 GB) lower peak GPU memory** than the base press, large, monotone, and *exact* for the KV cache (linear in kept length). This is the headline: the accuracy advantage of §4.1 is a memory advantage, same quality, much smaller footprint, which is exactly what lets a fixed GPU serve longer contexts or larger batches.

Decode speed is ~flat at this scale, and that is expected. At 8B / 16k / batch 8 the per-token decode is *weight-bound*, not KV-bound: reading the 16 GB of bf16 weights dominates each step, so shrinking the cache barely moves throughput/latency and the iso-accuracy differences (± 5 %) sit within run-to-run noise (the -5 %/ -6 % at the 50 % target is a single noisy point; Fig S5). Decode *does* speed up going from a full to a compressed cache ($156.7 \rightarrow \sim 185$ tok/s, **+18 %**; $50.7 \rightarrow \sim 42$ ms p50), but at iso-accuracy *both* presses operate compressed, so the gap between them is small. The throughput/latency win grows as decode becomes KV-bound (longer context, larger batch, or a larger model where the KV cache is a bigger share of each step), while the **memory** saving holds at every scale (e.g. full-cache 32 k = 39 GB $\rightarrow$ 33 GB at $r=0.75$, batch-independent). So *at the same output quality* our press runs on a much smaller cache: a large, immediate memory saving now, and throughput once decode is KV-bound.

Why the memory and speed columns trend opposite. The memory saving is largest at the 40 % target (4.5×, 78 %) yet the speed gain is largest at the 80 % target, seemingly backwards. It follows from the shapes of the two curves. *Memory* is **linear** in cache length, so its saving tracks the compression *ratio*, which is biggest when the target accuracy (hence the kept cache) is low. *Latency/throughput* are **non-linear**: they are flat until a **KV-bound “knee”** (~ 13 k kept tokens here, ≈ 79 % of a 16k context; Fig S5) and only move above it, where attention over the cache finally rivals the cost of reading the weights. Speed therefore improves only when a press’s operating point crosses that knee, and the 80 % target is the one case where the *baseline* must keep enough cache (~ 14 k tokens) to sit *above* the knee (slow), while ours (~ 8.7 k) stays below it (fast); at the lower targets *both* presses already operate below the knee, so the extra reduction shows up purely as memory, not speed. In one line: **memory follows the ratio; speed follows position relative to the knee.**

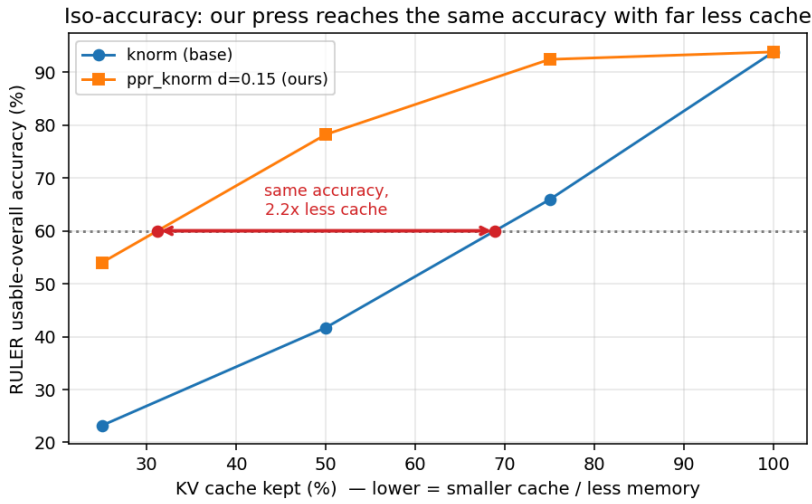


Figure 5: For a target accuracy (e.g. 60 %) the base press must keep ~69 % of the cache while ours needs only ~31 %, 2.2× less cache at equal accuracy.

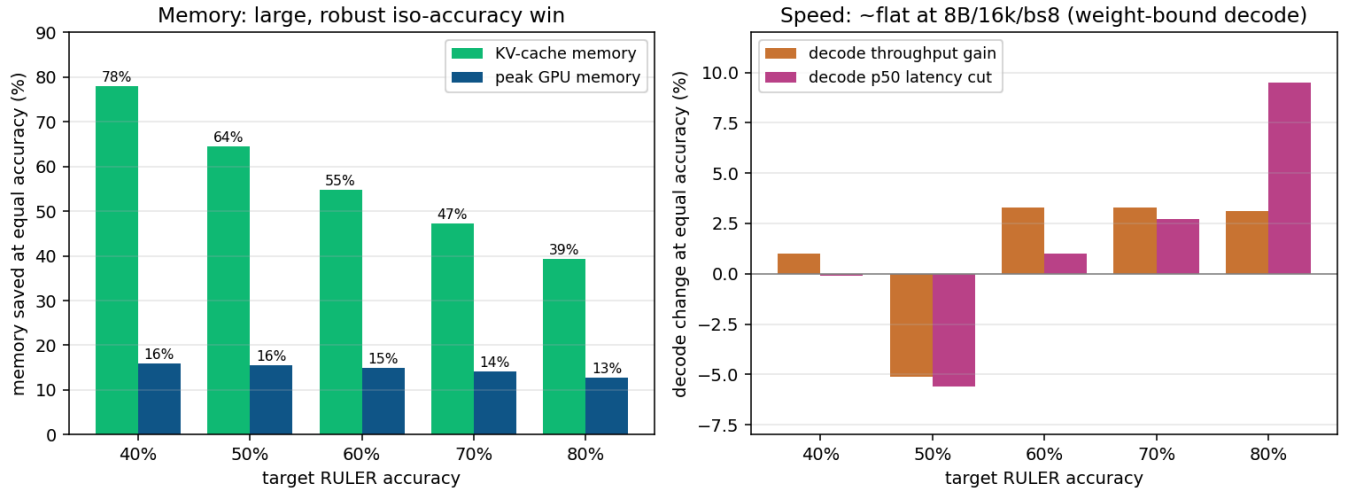


Figure 6: Systems gains at equal accuracy (16k ctx, batch 8, direct measurement). Left, memory saved: large and robust (KV cache 39–78 %, peak GPU 13–16 %). Right, decode throughput/latency change: ~flat (± 5 %, within noise) because decode is weight-bound at 8B/16k/bs8; the speed win materializes in more KV-bound regimes.

4.6 GraphKV comparison (additional benchmark)

The natural relational alternative to reinforcement is **suppression**: CentralityPress *adds* $\text{damping} \cdot A \cdot c$ to keep a corroborated core, whereas a GraphKV-style press (arXiv:2509.00388) *multiplicatively decays* each token by similarity to the top sources, $s_i = p_i \cdot \prod_{j \neq i} (1 - \text{relu}(\cos(k_i, k_j)))$, keeping a spread-out, diverse set. We compare against it on the *same* base press and graph. It is a **comparison benchmark**, **not part of the shipped press**; its code, tests and a runtime-injection runner live in `project/additional_benchmarks/` (see that folder’s README.md).

Head-to-head, reinforcement wins overall (paired per-example deltas of ppr_knorm d=0.15 vs graphkv_knorm; * = 95 % bootstrap CI excludes 0):

benchmark	c=0.25	c=0.5	c=0.75
RULER macro (13-task)	+12.3*	+27.8*	+21.8*
LongBench multi-hop F1	n/a	+0.8 (ns)	+9.4*

Our press beats the suppressor on RULER at every ratio; on real multi-hop QA the margin **grows with compression** (+9.4* at 0.75). It is also *cheaper*, the low-rank kernel adds +0.18 s prefill vs the suppressor’s +0.38 s (§4.4 workload), at identical decode cost.

Hypotheses H2 and H3 (reinforcement vs. suppression), with honest nuances:

- **H2 (reinforcement beats suppression on multi-hop): MIXED.** Supported on LongBench and RULER overall, but *refuted* on RULER’s variable-tracking vt: at $c=0.75$ the suppressor wins (graphkv 100 vs ppr 93.1, paired **-6.9***). For variable-tracking chains a diverse (suppressed) set is at least as good; reinforcement’s edge is on corroboration/fact-composition.
- **H3 (suppression $\geq$ reinforcement on single-needle): REFUTED.** Low-damping reinforcement *improves* single-needle (+29.0* vs base at 0.5) while suppression *hurts* it (-18.7*); the controlling variable is damping, not reinforcement per se.

In context. Beating the GraphKV suppressor is a lift over a relational baseline; independently, CentralityPress is a **ranked entry on the public KVPress leaderboard**, mid-pack overall, with a genuine matched-ratio 1st on fwe aggregation (§4.1). The method thus both wins its head-to-head against the suppression alternative and stands as a legitimate ranked board method.

Prior art, GraphKV’s own ablation. GraphKV (arXiv:2509.00388) itself compared its decay against a reinforcement variant (their Table 2, 128-token budget): decay 39.84, *enhanced* (reinforcement) 38.67, baseline 38.61, in their setup suppression edged out reinforcement and reinforcement added only ~ 0.06 over baseline. Our RULER result is the opposite (reinforcement $>$ suppression); the likely reason is the **personalized teleport**, our reinforcement is anchored to a base press that keeps the outlier needle, whereas an un-anchored “enhanced” propagation localizes on the dense core (the failure our centrality_pure ablation shows). We flag the disagreement rather than obscure it.

5. Discussion

Why a little reinforcement helps a lot. A single retrieval/answer token is an outlier; the base press may rank it well but ranks its *supporting context* poorly, so under compression the neighbours are evicted and the model loses the corroboration it needs to use the needle. A small damping propagates importance from the (teleport-anchored) needle to its graph neighbours, keeping the needle *and* its support. This is why $d=0.15$ recovers most of the full-cache accuracy while the base press alone does not.

Why too much reinforcement collapses. As damping $\rightarrow 1$ the teleport anchor weakens, centrality localizes on the densest (most redundant) cluster, and the outlier needle, low centrality by construction, is evicted. $d \geq 0.5$ and pure ($d=1$) collapse to near-zero on single-needle. The clean unimodal damping curve (base at $d=0$, peak at $d \approx 0.15$, collapse by $d=0.5$) is the method’s empirical signature.

A negative finding: SnapKV as a teleport base. SnapKV force-sets its recent-window scores to the maximum; z-scoring then makes the teleport pile onto the last tokens and starve mid-context needles, so centrality_ppr_snapkv collapses even with standardize_teleport. KnormPress (smooth, no spike) is the recommended base.

Systems trade-off. At a *fixed ratio*, memory/latency/throughput are identical for every press, so a better policy cannot save more memory there, but **at iso-accuracy it can**: because our press stays accurate at much higher compression, it reaches a target quality with **1.6 $\times$ –4.5 $\times$ less KV cache and ~ 13 –16 % (~ 6 GB) lower peak GPU memory** (§4.5), converting the accuracy edge into a real memory saving. The PageRank iterations add a small, bounded prefill cost ($O(T \cdot S \cdot \text{head_dim})$, $\sim 5\%$ here) and nothing at decode. Decode *throughput/latency* are $\sim$ flat at iso-accuracy on 8B/16k/bs8 because decode there is weight-bound (the per-step cost is dominated by reading the model weights, not the cache); the speed win emerges once decode is KV-bound, longer context, larger batch, or a larger model, while the memory saving is immediate at every scale.

Limitations. (1) One 8B model on one GPU; larger models / 32k context / Qwen replication are future work. (2) The RULER *table* numbers are a **fraction-0.06 screen** ($\approx 30/\text{task}$, **sdpa**); the leaderboard-eligible version is the **full-fraction flash_attention_2** board-grid run, which reproduces the screen (§4.1). (3) Absolute cross-method standing is modest (mid-pack on retrieval; a matched-ratio edge only on fwe aggregation); the large numbers are lifts *over the base press*, not over the field. (4) The LongBench gain at 0.5 is not significant, and H2 is mixed (refuted on RULER vt, §4.6); the method’s value is clearest under aggressive compression and on aggregation/corroboration. (5) All systems measurements use HuggingFace transformers; under a paged allocator (vLLM / PagedAttention) a page is reclaimed only when every slot in it is free, so scattered eviction would not free memory proportionally and the iso-accuracy memory saving would not transfer directly.

6. Conclusion and future work

A training-free personalized-PageRank eviction scorer, anchored to a cheap base press and using an $O(S \cdot \text{head_dim})$ low-rank kernel, **significantly and consistently beats its base press** on RULER retrieval/corroboration (all ratios) and on LongBench multi-hop QA (under aggressive compression), and also beats a GraphKV-style suppression baseline (§4.6); a damping=0 floor recovers the base press’s ranking (empirically within noise of the base, $\Delta \approx 0$). It is a **ranked entry on the public KVPress leaderboard**, mid-pack overall, with a matched-ratio 1st on fwe aggregation (§4.1). At **iso-accuracy** it runs on **1.6×–4.5× less KV cache and ~13–16 % lower peak GPU memory** than the base press, the accuracy win re-expressed as a memory saving, with throughput/latency gains that grow once decode becomes KV-bound. The mechanism is *bounded* reinforcement (small damping); unbounded reinforcement (pure centrality) reproduces the classic outlier-eviction failure.

Future work: head-adaptive damping (wrap with AdaKVPress); a combined reinforce-then-decorrelate press (keep central *and* diverse); pre-RoPE graphs; larger models and 32k context; a chat-templated RULER to expose the multi-hop/aggregation tasks; and an upstream kvpress leaderboard submission.

Reproduction: `project/reproduction/README.md` (env, commands), `project/reproduction/scripts/sweep_longbench.py` (sweeps), `project/reproduction/analysis/analyze_paired.py` (paired stats), `project/reproduction/results/` (per-run `config.yaml` + `metrics.json`). The press lives in `kvpress/presses/centrality_press.py`; see `project/reproduction/README.md` for the number $\rightarrow$ run-directory map.